demo

October 20, 2026

1 Demo: CRF Model

The following is a simplified version of what you will be implementing in this week's assignment

[1]:
```
import sklearn_crfsuite
from sklearn_crfsuite import metrics
from sklearn.model_selection import train_test_split

X = [['777', 'Brockton', 'Avenue', 'Abington', 'MA', '2351'],
  ['30', 'Memorial', 'Drive', 'Avon', 'MA', '2322'],
  ['250', 'Hartford', 'Avenue', 'Bellingham', 'MA', '2019'],
  ['700', 'Oak', 'Street', 'Brockton', 'MA', '2301'],
  ['66-4', 'Parkhurst', 'Rd', 'Chelmsford', 'MA', '1824'],
  ['591', 'Memorial', 'Dr', 'Chicopee', 'MA', '1020'],
  ['55', 'Brooksby', 'Village', 'Way', 'Danvers', 'MA', '1923'],
  ['137', 'Teaticket', 'Hwy', 'East', 'Falmouth', 'MA', '2536'],
  ['42', 'Fairhaven', 'Commons', 'Way', 'Fairhaven', 'MA', '2719'],
  ['374', 'William', 'S', 'Canning', 'Blvd', 'Fall', 'River', 'MA', '2721']]

y = [['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
  ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
 ␣
 ↪['address','address','address','address','address','city','city','state','zip']]

def token_features(token):
    return {
        "word": token,
        "is_lowercase": token.isdigit()
    }

X_tokenized = []
```

```
for seq in X:
    new_X = []
    for token in seq:
        new_X.append(token_features(token))
    X_tokenized.append(new_X)

# Here, we convert each token in X to it's feature dictionary returned by␣
 ↪token_features
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)

crf = sklearn_crfsuite.CRF()

crf.fit(X_train, y_train)

y_pred = crf.predict(X_test)

accuracy = metrics.flat_accuracy_score(y_test, y_pred)

print(accuracy)
```

```
1.0
```

[ ]: